

Security in Software Applications

Security Engineering for Software



SAPIENZA
UNIVERSITÀ DI ROMA

Daniele Friolo

friolo@di.uniroma1.it

<https://danielefriolo.github.io>

Definitions

- *Software Engineering*: Concept of creating and **maintaining** software applications by applying **technologies** and **practices** from computer science and project management fields [Wikipedia]
- Secure Software Engineering

“Current”/Traditional Software Engineering

- Over 30 years of software development experience created a well defined application software development lifecycle



- There are many software development methodologies (ex. XP, waterfall, etc) they all have these basic steps
- Capability Maturity Model for Software (SW-CMM) is used to measure quality of methodologies employed

Motivation

- This application development process in its essence fails to address security issues
- Consequently, security flaws are identified only at the later stages of the application lifecycle. And thus
 - Much greater cost to fix
 - High maintenance cost
 - ...
- Nearly every company/organization utilizes network security infrastructure (e.g. Firewalls, IDS, etc)
- But very small number of them invest in application security strategy, design, and code review services

So

- For the software industry, the key to meeting demand for improved security is to implement **repeatable** processes that **reliably** deliver measurably improved security
- Thus, there must be a transition to a more stringent software development process that greatly focuses on security
- Goal: **minimize** the number of security vulnerabilities in design, implementation, and documentation
 - Identify and remove vulnerabilities in the development lifecycle **as early as possible!!!**

Building Secure Software

Three essential components

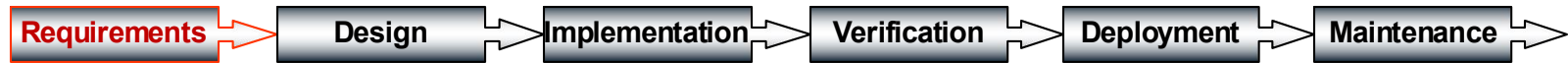
- Repeatable process
- Engineer Education
- Metrics and Accountability
- **SDL – Secure Development Lifecycle**
 - Used along with traditional/current software development lifecycle/techniques in order to introduce security at every stage of software development

Building Secure Software

Three essential components

- Repeatable process
- Engineer Education
- Metrics and Accountability
- **SDL** – Secure Development Lifecycle
 - Used along with traditional/current software development lifecycle/techniques in order to introduce security at every stage of software development

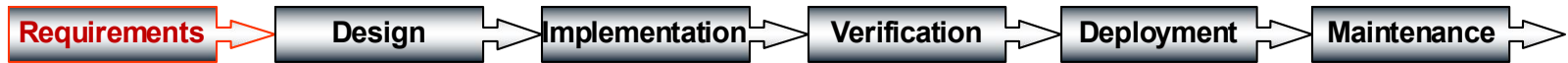
SDL – Requirements Phase



- Development of requirements
 - Gather information about application [customer/experience/survey]
- Analysis of requirements
 - Are all the security issues addressed [C, I, A, ...]
- Verification of requirements
 - Any inconsistencies? - system interface - correctness
 - Documentation !!!
 - Feasibility of requirements
- [repeat]

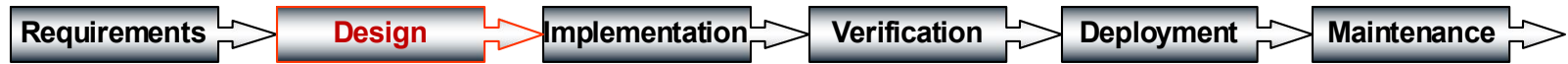
Planning at this stage offers the best opportunity to build secure software in the most efficient manner [cost, time, etc]

SDL – Requirements Phase



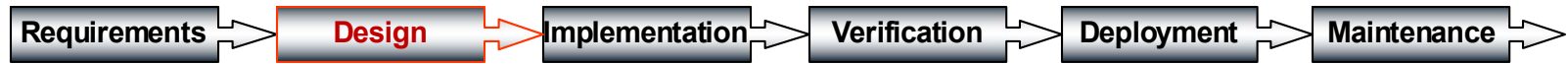
- Develop Security Requirements
 - Security Requirements of a system/application must be developed **along with** any other requirements (e.g. functional, legal, user, etc)
- Risk analysis
 - Identify all the assets at risk
 - Identify all the threats
- Develop security **policies**
 - Used as guidelines for requirements
- Develop security **metrics**

SDL – Design Phase



- At this stage, all design decisions are made, about
 - Software Architecture
 - Software components
 - Programming languages
 - Interfaces
 - ...
- Develop documentation
- Confirm that all requirements are followed and met

SDL – Design Phase



- Threat Models
- Input Data Types
- Security Use Cases
- Security Architecture

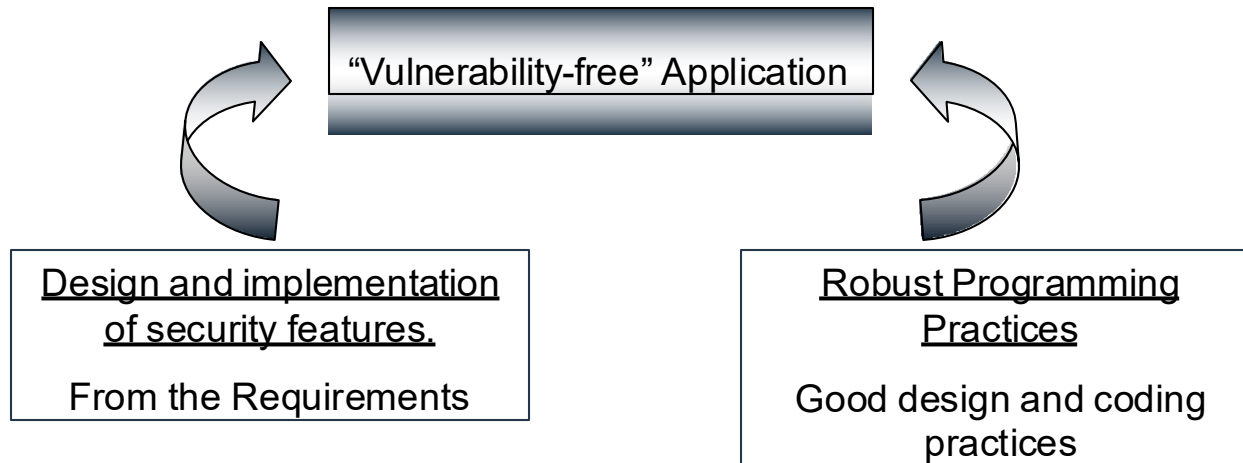
- Defense in Layers/Separate Components/Least Privilege
- Tool
 - SecureUML and UMLsec

SDL – Implementation Phase

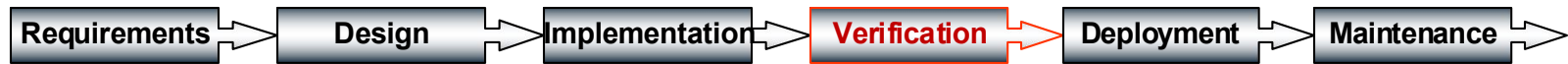


- This is the stage where coding is done.
- To produce secure software
 - Coding Standards
 - Centralized Security Modules
 - Secure builds and configurations
 - Known security vulnerabilities - use good programming practices.
Be aware of
 - Race conditions
 - Buffer overflow
 - Format string
 - Malicious logic
 - ...
- Follow Design and Develop Documentation [further]

SDL – Implementation Phase

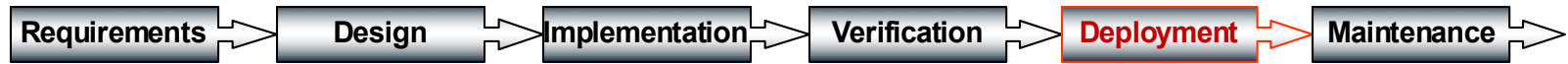


SDL – Verification Phase



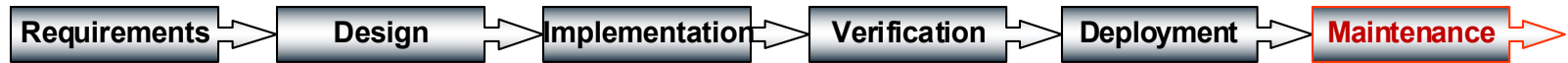
- Testing of the code developed in the previous stage
- Cleared security tests
- Security vulnerability tracking
- Code Reviews
- Documentation

SDL – Release Phase



- Secure Management Procedures
- Monitoring Requirements
- Security Upgrade Procedures

SDL – Response Phase



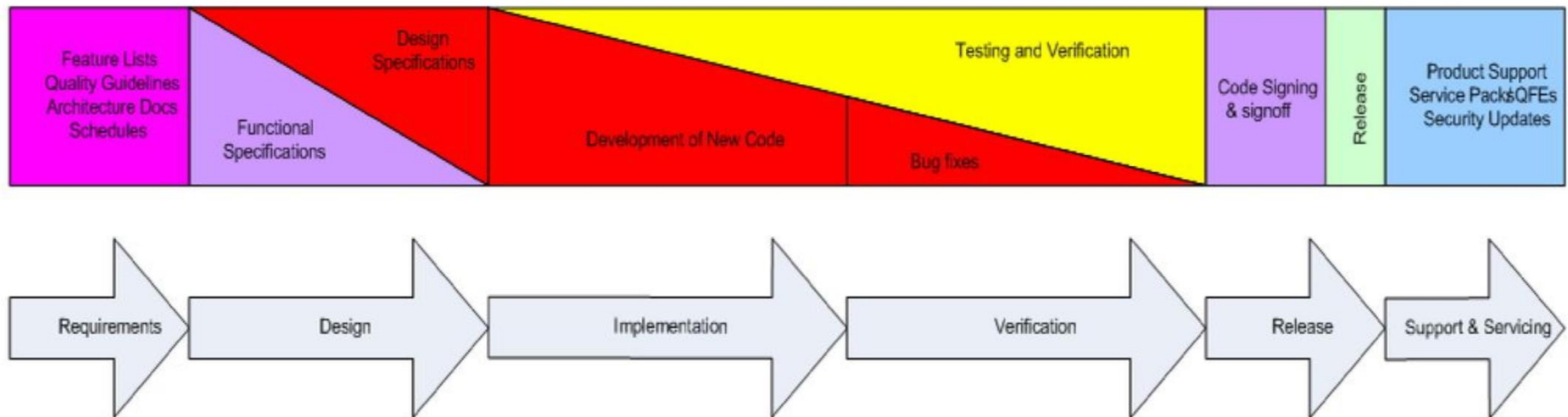
- Causes:
 - Customer feedback
 - Security incident details and vulnerability reports
 - ...
- Types of maintenance
 - Need to introduce new functionality
 - Need to upgrade to keep up with technology
 - Discovered vulnerability

Facts

- Every security vulnerability / flaw overlooked in an earlier phase will show-up at later phase[s]
- Resulting into greater
 - Cost
 - Timeof the software development and/or maintenance

Microsoft Trustworthy Computing SDLC

- Generally accepted SDL process at MS
- (actually spiral not “waterfall” as it indicates)



Microsoft – Case Study SD³ + C

■ Secure by Design

- Software designed and implemented to “protect” itself and its information

■ Secure by Default

- Accept the fact that software will not achieve perfect security
- To minimize the harm when vulnerabilities exploited, software’s default state should promote security (e.g. least necessary privileges)

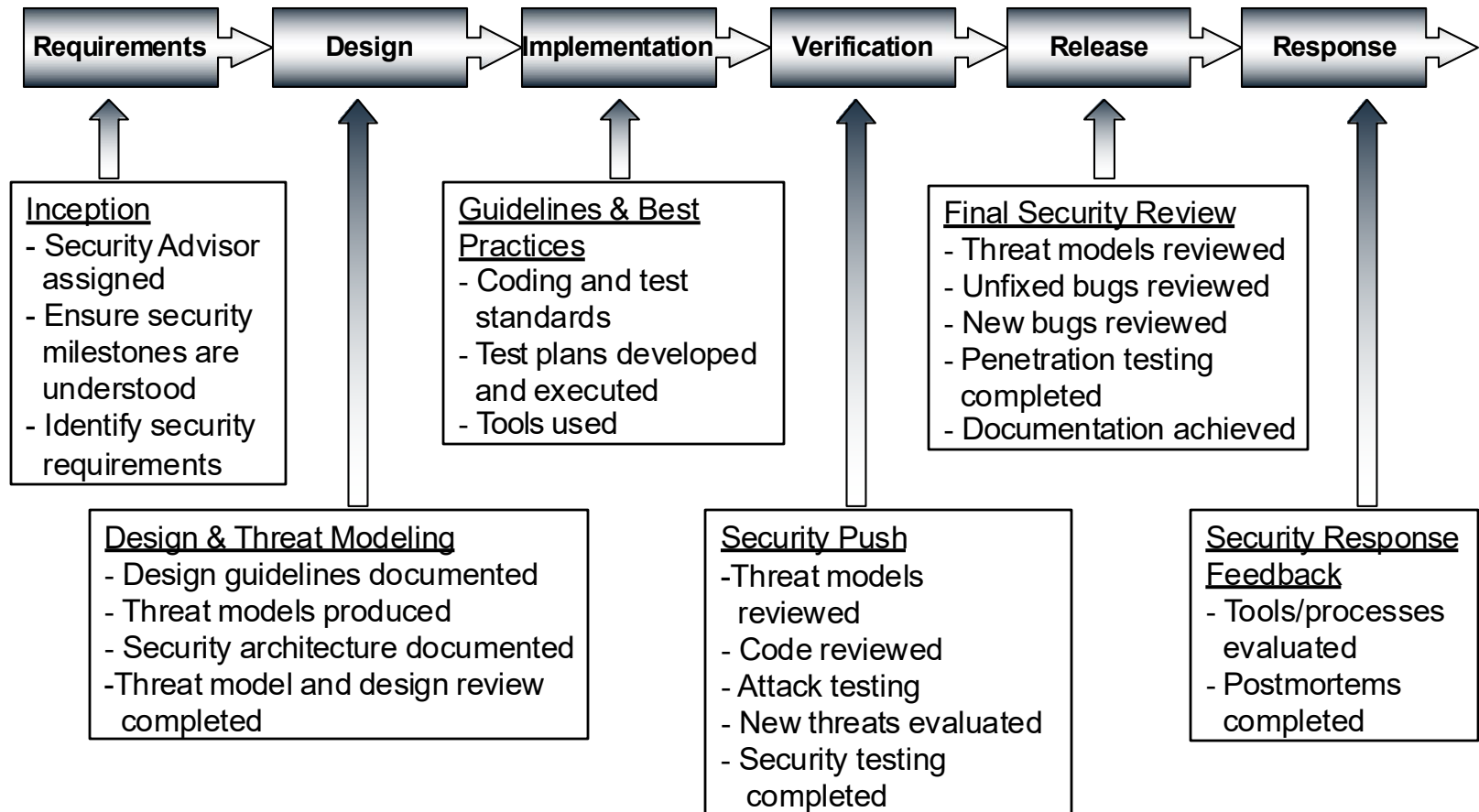
■ Secure in Deployment

- Software accompanied by tools and guidance to assist secure use

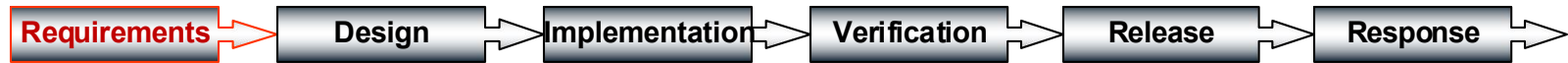
■ Communications

- Developers should be prepared for discovery of product vulnerabilities and should communicate openly and responsibly with end users. (e.g. patching, deploying workarounds)

SDL @ Microsoft

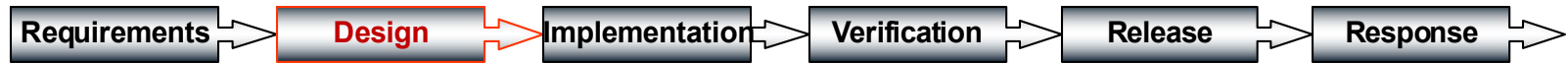


SDL – Requirements Phase



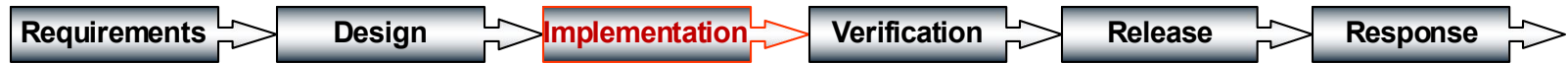
- Product and central security teams assign “security buddy” – security advisor
 - Point of contact / resources / guide
 - Review plans / recommendations / resources
- Product team considers
 - How security will be integrated into the development process
 - Key security objectives
- Documentation

SDL – Design Phase



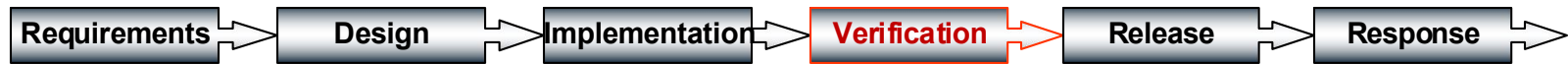
- Define security architecture and design guidelines (identify TCB, use layering)
- Document the elements of the software attack surface
- Conduct threat modeling
- Define supplemental ship criteria

SDL – Implementation Phase



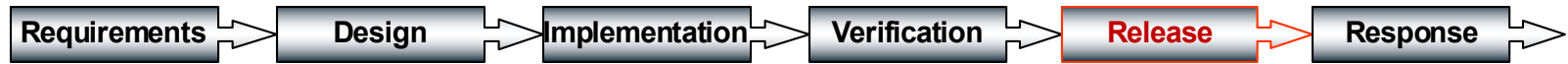
- Apply coding and testing standards
- Apply security testing tools, including fuzzing tools
 - Supplies structured but invalid inputs
- Apply static-analysis code scanning tools
- Conduct code reviews

SDL – Verification Phase



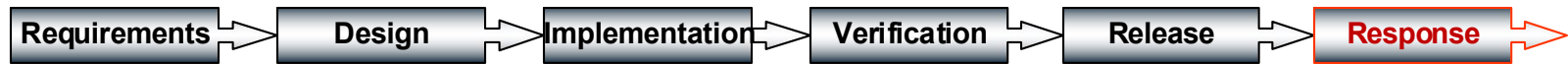
- “Beta” testing stage
- “Security push”
 - security code reviews beyond those completed in implementation phase (e.g. legacy code not modified)
 - Testing of high priority code
 - Trying to “break” the code

SDL – Release Phase



- During the release, software is subject to Final Security Review [FSR]
- The goal of FSR is to determine whether, from security viewpoint, the software is ready to be delivered to costumers
- Not *pass / fail*
- Goal is to find every remaining security vulnerability in software
 - If found, revisit all the preceding phases and fix the root problem
- Conducted by central security team

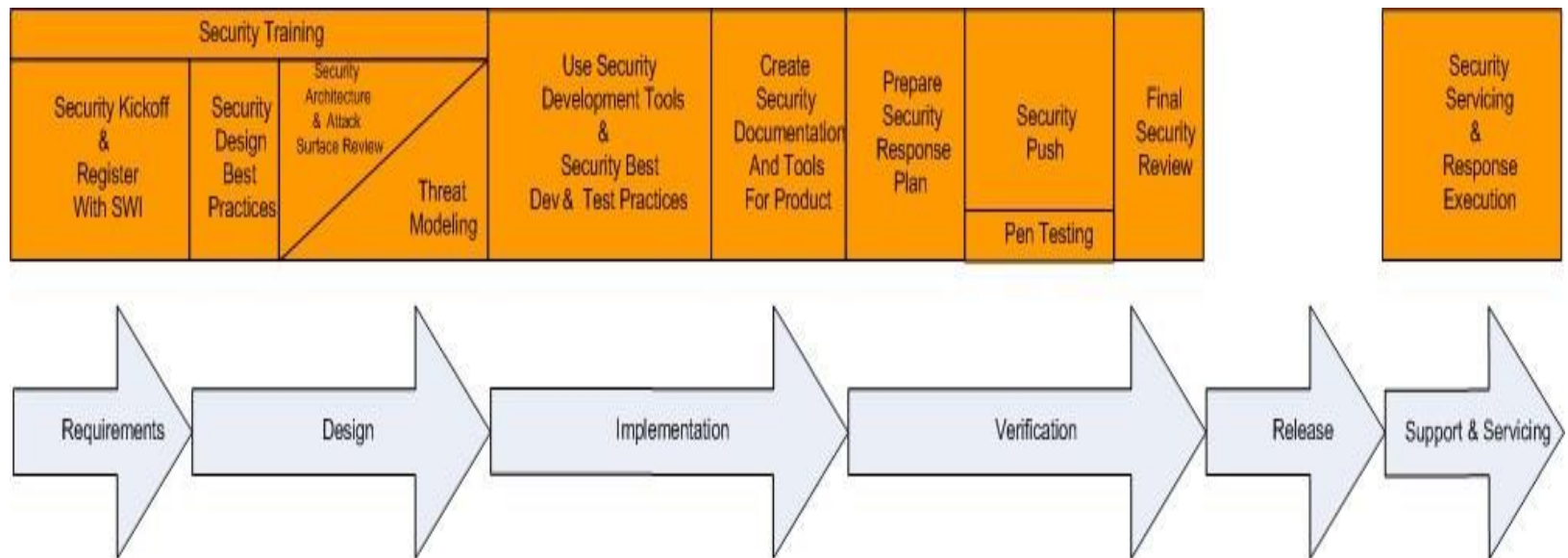
SDL – Response Phase



- Despite use of SDL, resulting software is not vulnerability free; and even if it could be so, new attacks would be possible
- Evaluation of reports
- Development of patches and security updates

SDL @ Microsoft

- After adding the SD³+ C paradigm



- Mandatory Application of the SDL
- Mandatory Education
- Metrics for Product Teams
- The Central Security Team

Results

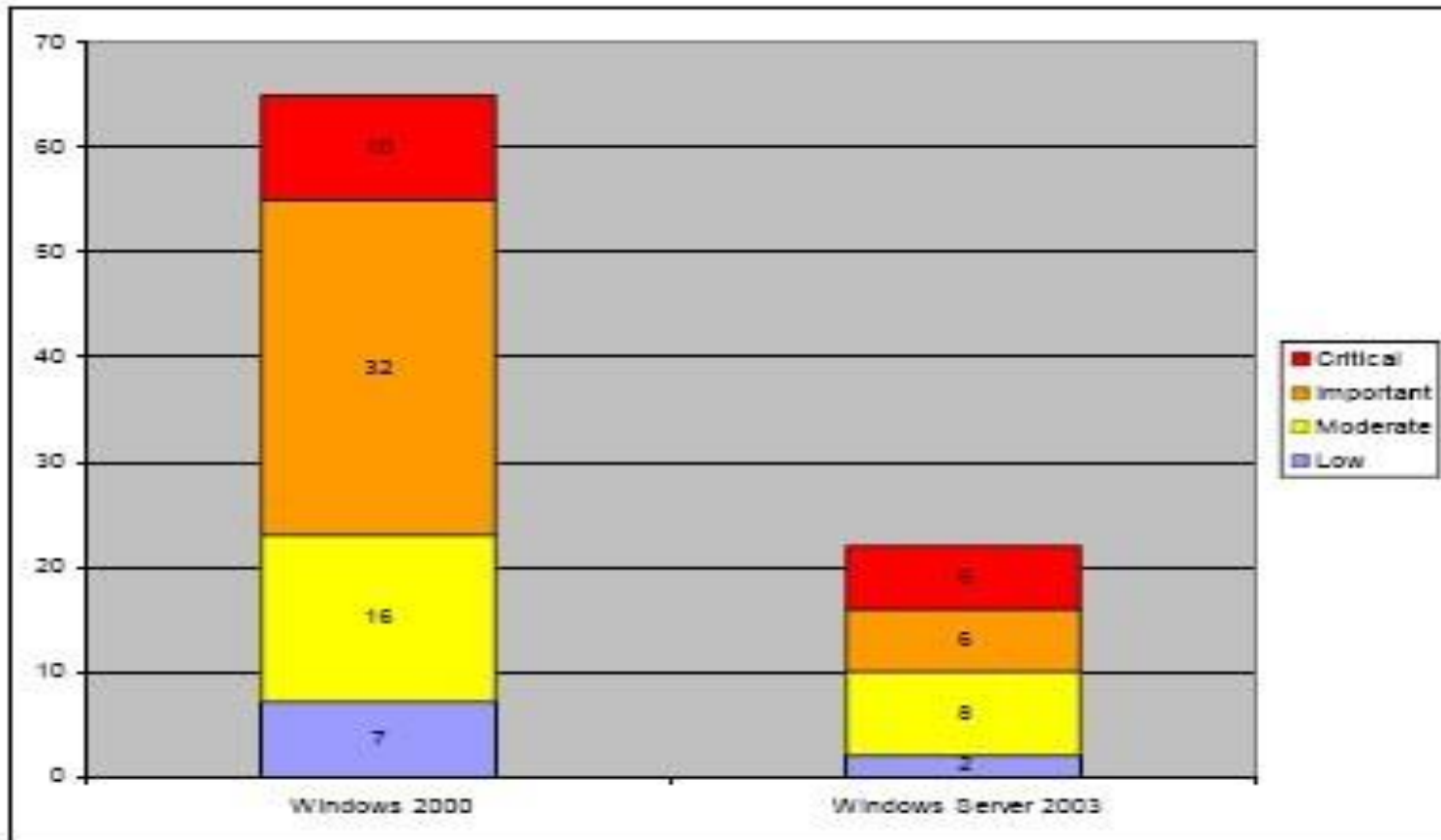


Figure 3. First Year Security Bulletins: Windows 2000 vs. Windows Server 2003